



APACHE MAVEN

гл.ас. Пенчо Малинов



КАКВО Е MAVEN? ОСНОВНИ ЦЕЛИ

- Maven е инструмент за изграждане и управление на всички Java базирани проекти.
- Основни цели - Основната цел на Maven е да позволи на разработчика да разбере цялостното състояние на усилията за развитие в най-краткия период от време. За да се постигне тази цел, има няколко области на безпокойство, които Maven се опитва да се справи с:
 1. Опростяване на процеса на създаване
 2. Осигуряване на еднакво изграждане система
 3. Осигуряване на качествена информация за проекта
 4. Осигуряване на насоки за развитие на най-добри практики
 5. Позволяване на прозрачна миграция към нови функции

ОПРЕДЕЛЕНИЕ

- **Maven е инструмент за управление на проекти, който включва обектен модел на проект, набор от стандарти, жизнен цикъл на проекта, система за управление на зависимости и логика за изпълнение на целите на приставките на определени фази от жизнения цикъл.**

ИСТОРИЯ

- Проект на Apache
 - Спонсориран главно от Sonatype
- Развитие през годините
 - Maven 1 (2003)
 - Много недоразвит
 - Използван в Stack 1
 - Maven 2 (2005)
 - Напълно пренаписан
 - Не поддържа предходната версия
 - Използва се в Stack 2.0,2.1,2.2,3.0
 - Maven 3 (2010)
 - Като Maven 2 но много по - стабилен
 - Използван в Stack 2.3, 3.1

ДРУГИ JAVA БАЗИРАНИ ИЗГРАЖДАЩИ ИНСТРУМЕНТИ

- Ant (2000)
 - Прародителя на инструментите за изграждане на Java проекти
 - Използва се XML за конфигурация
 - Много гъвкав
- Ant+Ivy (2004)
 - Ant но с Dependency Management
- Gradle (2008)
 - Комбинира Maven структура с Groovy синтаксис
 - Лесен за разширяване

ХАРАКТЕРИСТИКИ НА MAVEN

- Проста настройка на проекта, която следва най-добрите практики - започвате нов проект или модул за секунди
- Може лесно да работи с няколко проекта едновременно
- Превъзходно управление и автоматично актуализиране на зависимости(dependency) и транзитивни зависимости (transitive dependencies)
- Може лесно да работи с няколко проекта едновременно
- Предоставя голямо и нарастващо хранилище на библиотеки и метаданни
- Разширява се, с възможност за лесно написване на приставки в Java или скриптов езици
- Незабавен достъп до нови функции с малка или никаква допълнителна конфигурация
- Ant задачи за управление на зависимостта и разполагане извън Maven
- Изграждане (Build) базирано на модели: Maven е в състояние да изгради произволен брой проекти в предварително дефинирани изходни типове като JAR, WAR

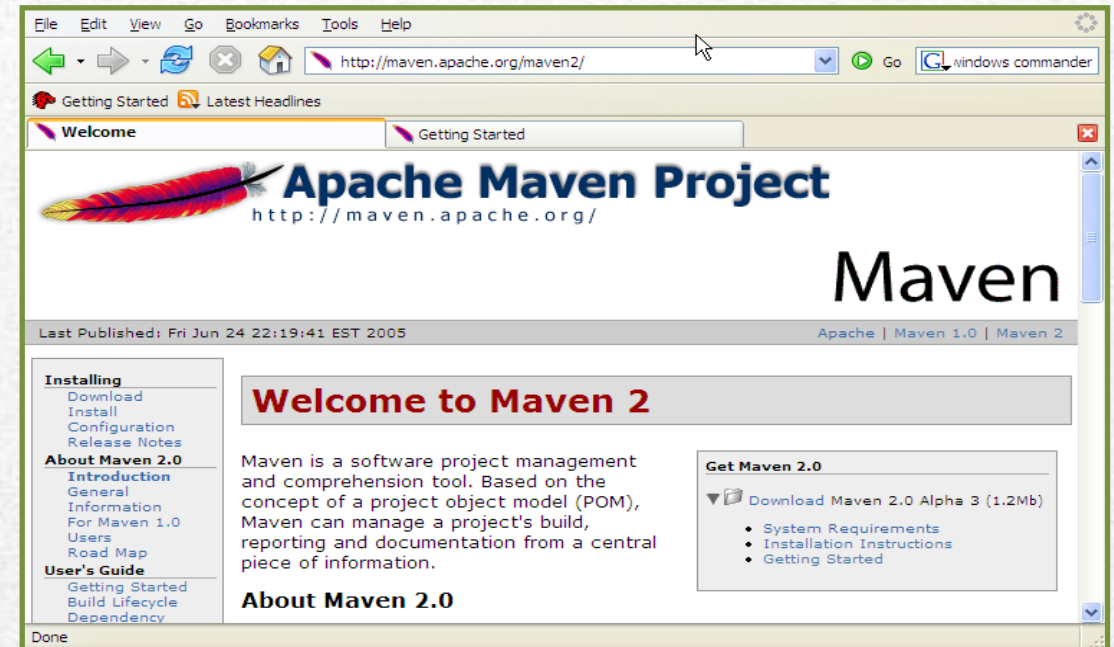
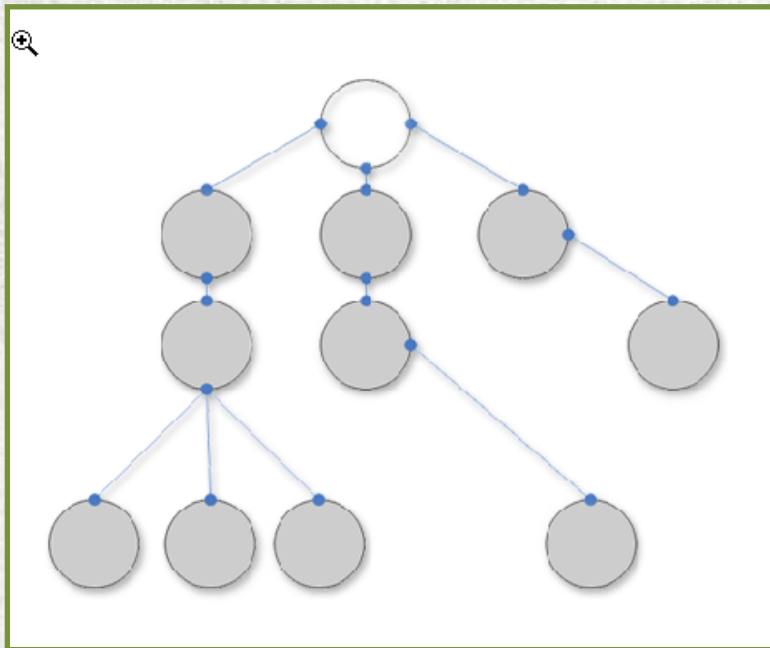
ХАРАКТЕРИСТИКИ НА MAVEN

- Съгласуван сайт за информация за проекта: Използвайки същите метаданни, както при процеса на изграждане, Maven може да генерира уеб сайт или PDF, включително всяка документация, която искате да добавите, и добавя към стандартните отчети за състоянието на разработката на проекта.
- Управление на версиите: Без много допълнителна конфигурация, Maven може да се интегрира с вашата система за управление на версиите (като Subversion или Git). Може също да доставя изградения проект в място за разпространение, което да се използва от други проекти. Maven може да публикува отделни изходни типове като JAR.
- Управление на зависимостите (Dependency management): Maven насърчава използването на централно хранилище на JAR и други зависимости. Maven идва с механизъм, който клиентите на проекта ви могат да използват, за да изтеглят всички JAR-ове, необходими за изграждането на вашия проект от централно хранилище. Това позволява на потребителите на Maven да използват повторно JAR-ове в рамките на проекти и насърчава комуникацията между проектите.

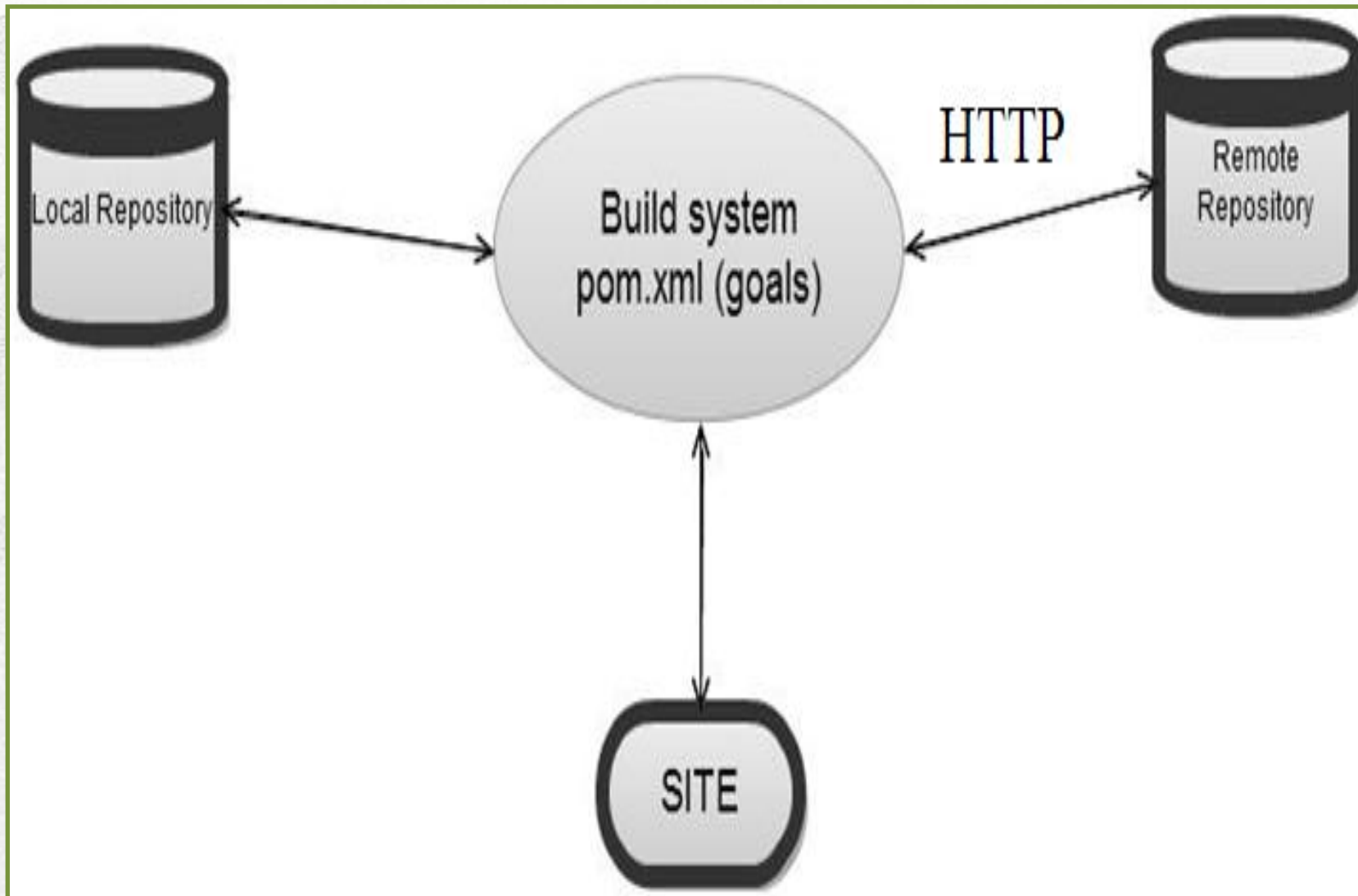
ОСНОВНИ ФУНКЦИОНАЛНОСТИ НА MAVEN

- Build-Tool
- Dependency Management Tool
- Documentation Tool

```
C:\WINDOWS\system32\cmd.exe
Downloading: http://repo1.maven.org/maven2/org/apache/maven/wagon/wagon/1.0-alpha-4/wagon-1.0-alpha-4.pom
3K downloaded
Downloading: http://repo1.maven.org/maven2/org/apache/maven/wagon/wagon-provider-api/1.0-alpha-4/wagon-provider-api-1.0-alpha-4.jar
45K downloaded
Downloading: http://repo1.maven.org/maven2/org/apache/maven/maven-artifact-manager/2.0-alpha-3/maven-artifact-manager-2.0-alpha-3.jar
32K downloaded
[INFO] [install:install]
[INFO] Installing C:\my-app\target\my-app-1.0-SNAPSHOT.jar to C:\Documents and Settings\Administrator\TOSHIBA\m2\repository\com\mycompany\app\my-app\1.0-SNAPSHOT\my-app-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESSFUL
[INFO]
[INFO] Total time: 47 seconds
[INFO] Finished at: Fri Jun 24 16:24:10 PDT 2005
[INFO] Final Memory: 2M/5M
[INFO]
```



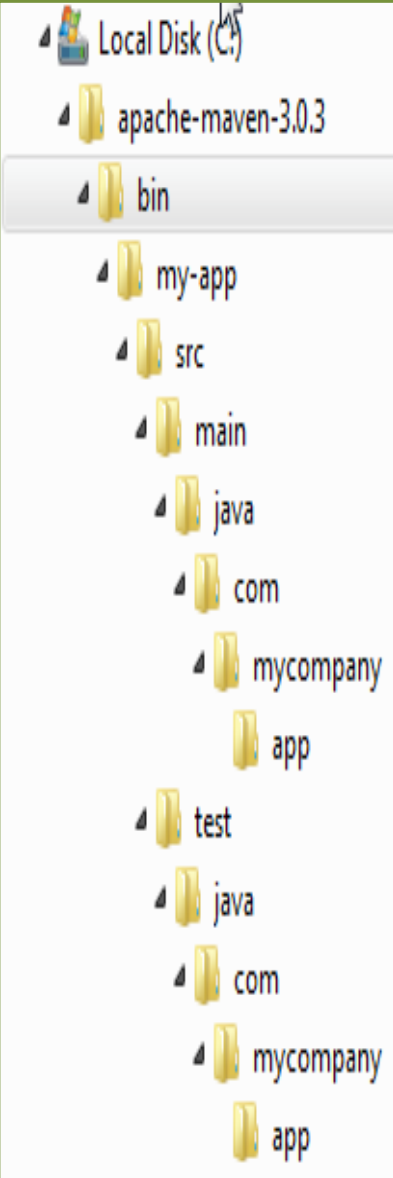
АРХИТЕКТУРА



CONVENTION OVER CONFIGURATION

- “Convention Over Configuration” е проста концепция. Системите, библиотеките и рамките трябва да приемат разумни стойности по подразбиране. Без да се изисква ненужна конфигурация, системите трябва “просто да работят”.
- Maven включва тази концепция, като осигурява разумно поведение по подразбиране за проектите. Без персонализиране се приема, че изходният код е в **`${basedir}/src/main/java`**, а ресурсите са в **`${basedir}/src/main/resources`**. Тестовите са в **`${basedir}/src/test`**, а проектът трябва да създаде **JAR** файл. Maven предполага, че искате компилираният байт код да е в **`${basedir}/target/classes`** и след това да създадете разпространен **JAR** файл в **`${basedir}/target`**.
- **Ако следвате конвенциите, Maven ще изисква почти нулеви усилия - просто поставете кода си в правилната директория, а Maven ще се погрижи за останалото.**

Стандартни директории

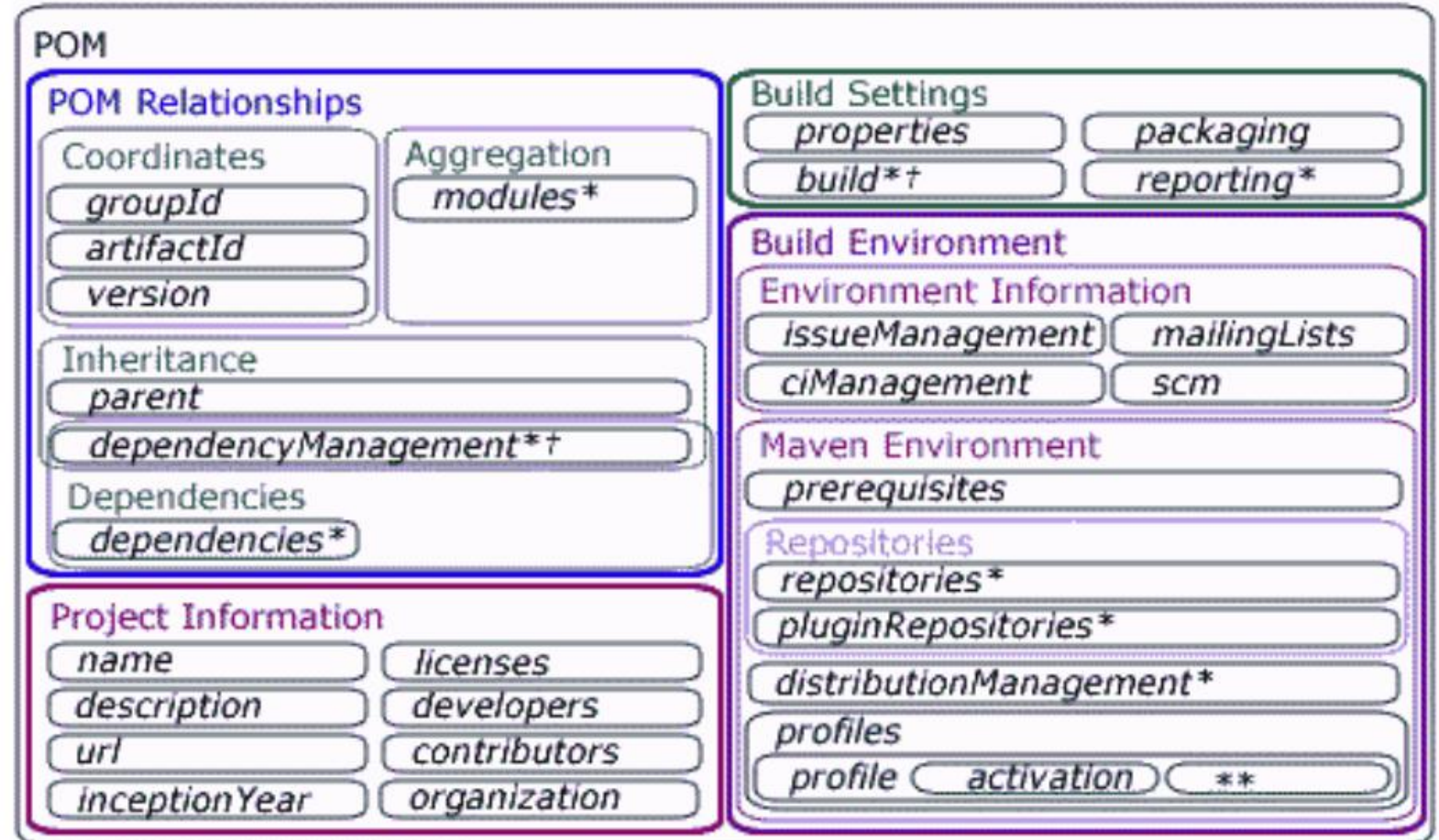
src/main/java	Application/Library sources	
src/main/resources	Application/Library resources	
src/main/filters	Resource filter files	
src/main/assembly	Assembly descriptors	
src/main/config	Configuration files	
src/main/scripts	Application/Library scripts	
src/main/webapp	Web application sources	
src/test/java	Test sources	
src/test/resources	Test resources	
src/test/filters	Test resource filter files	
src/site	Site	
LICENSE.txt	Project's license	
NOTICE.txt	Notices and attributions required by libraries that the project depends on	
README.txt	Project's readme	

POM - PROJECT OBJECT MODEL

- Maven проекти, dependencies, builds, artifacts: всичко това са обекти, които трябва да бъдат моделирани и описани. Тези обекти са описани в XML файл, наречен Project Object Model. POM казва на Maven с какъв вид проект се занимава и как да модифицира поведението по подразбиране, за да генерира изходен код. По същия начин уеб приложение на Java има web.xml, който описва, конфигурира и персонализира приложението. Проектът на Maven се дефинира от присъствието на pom.xml. Това е описателна декларация на проект за Maven.

POM - СТРУКТУРА

- Имаме 4 основни групи конфигурации:
 - Основни
 - Информация за проекта
 - Настройки за изграждането
 - Среда за изграждане



РОМ – СТРУКТУРА

ОСНОВНА КОНФИГУРАЦИЯ

- Идентификатори на проекта:
 - **groupId** – уникално поле на ниво организация или проект. Служи за приобщаване на проект към определена група. Примерно всички артефакти на Maven съществуват под **groupId org.apache.maven**.
 - **artifactId** – името на проекта. В комбинация с групата създават уникален идентификатор за проекта.
 - **version** – конкретната версия на проекта. В комбинация с предходните конкретизират не само проекта но и версията му.
 - **packaging** – определя изхода от процеса на изграждане на приложението. По подразбиране е JAR

```
1. <project xmlns="http://maven.apache.org/POM/4.0.0"
2.   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3.   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4.                       http://maven.apache.org/xsd/maven-4.0.0.xsd">
5.   <modelVersion>4.0.0</modelVersion>
6.
7.   <!-- The Basics -->
8.   <groupId>...</groupId>
9.   <artifactId>...</artifactId>
10.  <version>...</version>
11.  <packaging>...</packaging>
12.  <dependencies>...</dependencies>
13.  <parent>...</parent>
14.  <dependencyManagement>...</dependencyManagement>
15.  <modules>...</modules>
16.  <properties>...</properties>
17.
18.  <!-- Build Settings -->
19.  <build>...</build>
20.  <reporting>...</reporting>
21.
22.  <!-- More Project Information -->
23.  <name>...</name>
24.  <description>...</description>
25.  <url>...</url>
26.  <inceptionYear>...</inceptionYear>
27.  <licenses>...</licenses>
28.  <organization>...</organization>
29.  <developers>...</developers>
30.  <contributors>...</contributors>
31.
32.  <!-- Environment Settings -->
33.  <issueManagement>...</issueManagement>
34.  <ciManagement>...</ciManagement>
35.  <mailingLists>...</mailingLists>
36.  <scm>...</scm>
37.  <prerequisites>...</prerequisites>
38.  <repositories>...</repositories>
39.  <pluginRepositories>...</pluginRepositories>
40.  <distributionManagement>...</distributionManagement>
41.  <profiles>...</profiles>
42. </project>
```

РОМ – СТРУКТУРА

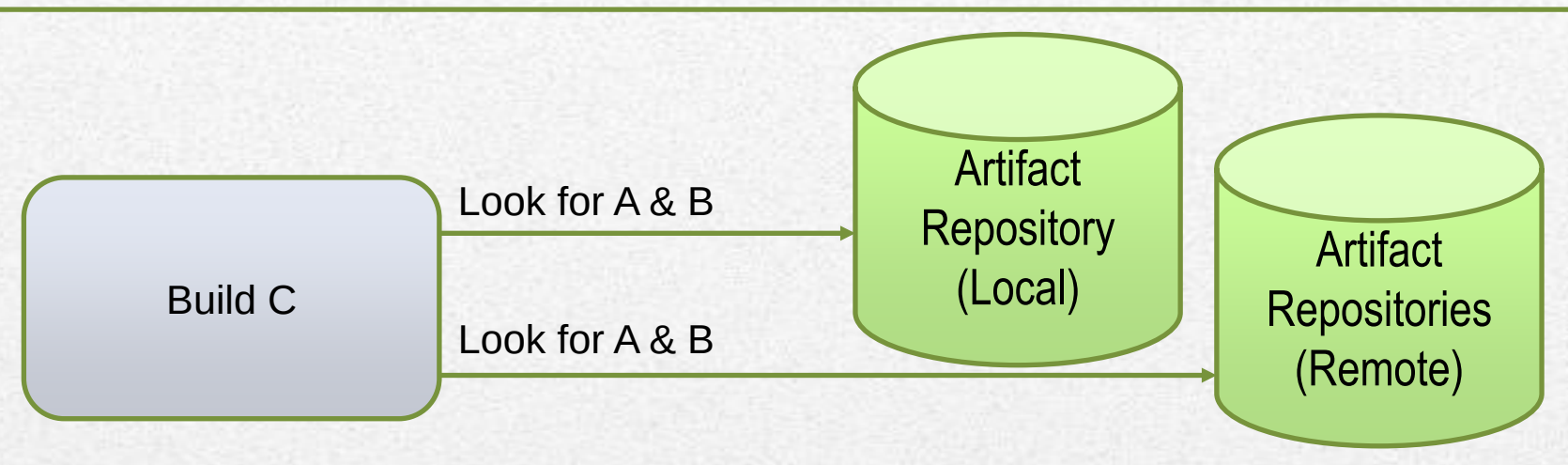
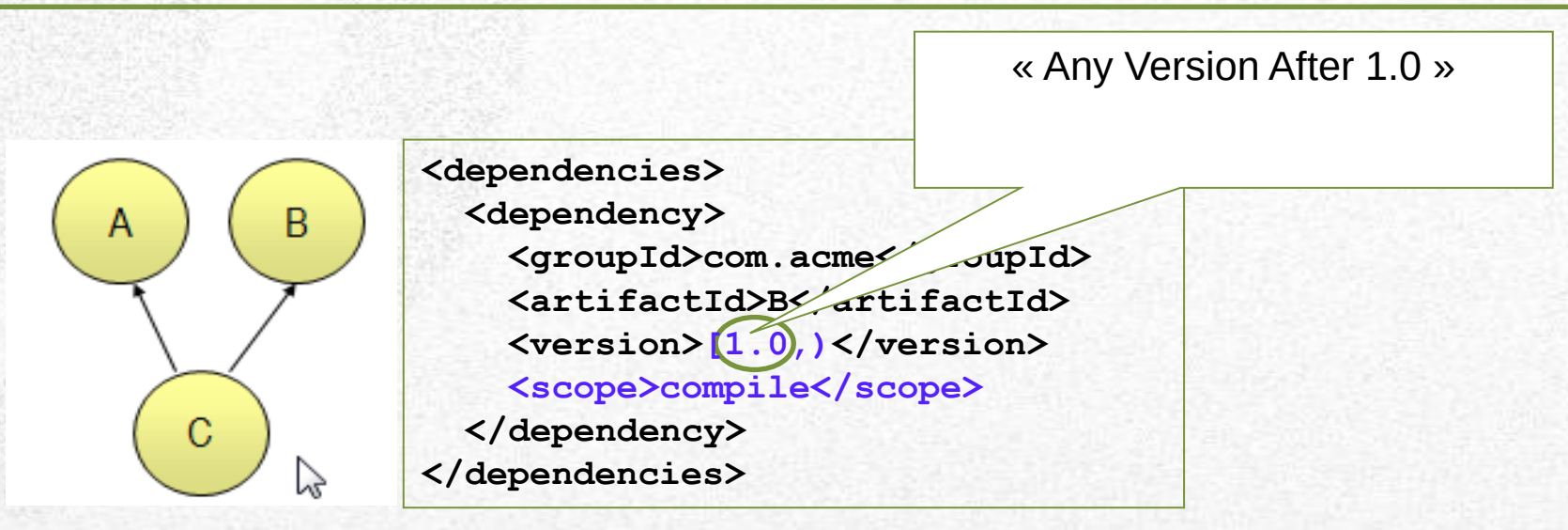
ОСНОВНА КОНФИГУРАЦИЯ

- Взаимоотношения – един от най-силните аспекти на Maven е управлението на взаимоотношенията. Тук се включват зависимости(dependencies, transitive dependencies), наследяване и обединение(много-модулен проект).
- Зависимости(dependencies) – Maven се грижи за управление на зависимостите. Той се грижи да изтегли всички описани зависимости и да ги предостави в проекта в желаната област на изпълнение и да изключи тези които не ни трябва

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.0</version>
    <type>jar</type>
    <scope>test</scope>
    <optional>true</optional>
  </dependency>
  ...
</dependencies>
```

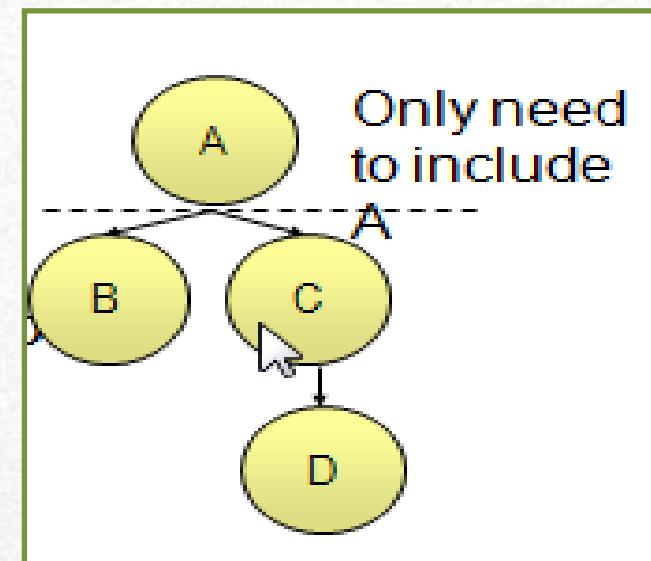
УПРАВЛЕНИЕ НА ЗАВИСИМОСТИ (DEPENDENCY MANAGEMENT)

- `1.0`: "Soft" requirement on 1.0
- `[1.0]`: "Hard" requirement on 1.0
- `(,1.0]`: $x \leq 1.0$
- `[1.2,1.3]`: $1.2 \leq x \leq 1.3$
- `[1.0,2.0)`: $1.0 \leq x < 2.0$
- `[1.5,)`: $x \geq 1.5$
- `(,1.0], [1.2,)`: $x \leq 1.0$ or $x \geq 1.2$
- `(,1.1), (1.1,)`: this excludes 1.1



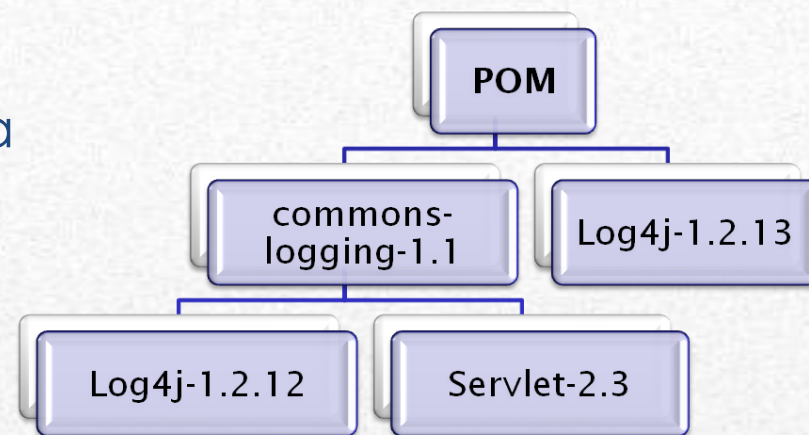
ТРАНЗИТИВНИ ЗАВИСИМОСТИ (TRANSITIVE DEPENDENCIES)

- Позволява да се избегне нуждата от откриване и определяне на библиотеки, които вашите собствени зависимости изискват – необходимо е само да се опише пряката зависимост в проекта и Maven ще се погрижи да добави нейните зависимости.
- Няма ограничения за нивата, от които могат да се събират зависимости.
- Проблем има само ако се открие циклична зависимост. Ако вашият проект зависи от A и A зависи от C и C зависи от B и B зависи от A.



УПРАВЛЕНИЕ НА ТРАНЗИТИВНИ ЗАВИСИМОСТИ

- **Dependency Mediation** - Тази функция определя коя версия на зависимост ще се използва, когато се срещат няколко версии на един артефакт. Винаги може да се гарантира версията, чрез изричното и упоменаване в POM проекта. Ако две версии на зависимости са в една и съща дълбочина в дървото на зависимостта, редът в декларацията има значение - първата декларация печели.
- **Dependency Management** – чрез тази функционалност могат да се упоменат версии на транзитивни зависимости.
- **Dependency Scope** – с тази функционалност се определя обхвата на действие
- **Excluded Dependencies** – при необходимост могат да се премахват транзитивни зависимости
- **Optional Dependencies** – когато се определи зависимост като „Optional“ тя няма да се добави като транзитивна



УПРАВЛЕНИЕ НА ЗАВИСИМОСТИ - ПРИМЕР

```
<project>
...
<dependencies>
  <dependency>
    <groupId>group-a</groupId>
    <artifactId>artifact-a</artifactId>
    <version>1.0</version>
    <exclusions>
      <exclusion>
        <groupId>group-c</groupId>
        <artifactId>excluded-artifact</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
  <dependency>
    <groupId>group-a</groupId>
    <artifactId>artifact-b</artifactId>
    <version>1.0</version>
    <type>bar</type>
    <scope>runtime</scope>
  </dependency>
</dependencies>
</project>
```

```
<project>
...
<dependencies>
  <dependency>
    <groupId>group-c</groupId>
    <artifactId>artifact-b</artifactId>
    <version>1.0</version>
    <type>war</type>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>group-a</groupId>
    <artifactId>artifact-b</artifactId>
    <version>1.0</version>
    <type>bar</type>
    <scope>runtime</scope>
  </dependency>
</dependencies>
</project>
```

УПРАВЛЕНИЕ НА ЗАВИСИМОСТИ - ПРИМЕР

```
<project>
...
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>group-a</groupId>
      <artifactId>artifact-a</artifactId>
      <version>1.0</version>

      <exclusions>
        <exclusion>
          <groupId>group-c</groupId>
          <artifactId>excluded-artifact</artifactId>
        </exclusion>
      </exclusions>
    </dependency>

    <dependency>
      <groupId>group-c</groupId>
      <artifactId>artifact-b</artifactId>
      <version>1.0</version>
      <type>war</type>
      <scope>runtime</scope>
    </dependency>

    <dependency>
      <groupId>group-a</groupId>
      <artifactId>artifact-b</artifactId>
      <version>1.0</version>
      <type>bar</type>
      <scope>runtime</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
</project>
```

```
<project>
...
<dependencies>
  <dependency>
    <groupId>group-c</groupId>
    <artifactId>artifact-b</artifactId>
    <!-- This is not a jar dependency, so we must specify type. -->
    <type>war</type>
  </dependency>

  <dependency>
    <groupId>group-a</groupId>
    <artifactId>artifact-b</artifactId>
    <!-- This is not a jar dependency, so we must specify type. -->
    <type>bar</type>
  </dependency>
</dependencies>
</project>
```

```
<project>
...
<dependencies>
  <dependency>
    <groupId>group-a</groupId>
    <artifactId>artifact-a</artifactId>
  </dependency>

  <dependency>
    <groupId>group-a</groupId>
    <artifactId>artifact-b</artifactId>
    <!-- This is not a jar dependency, so we must specify type. -->
    <type>bar</type>
  </dependency>
</dependencies>
</project>
```

DEPENDENCY SCOPE

- **compile-** Обхват на действие по подразбиране. Зависимостта е достъпна навсякъде в приложението.
- **provided-** Използва се, когато JDK или web контейнера ще осигури зависимостта по време на работа. Налична е по време на компилиране и изграждане на приложението
- **runtime-** Този обхват на достъпност се използва когато зависимостта не е необходима по време на компилиране а само по време на работа на приложението.
- **test-** Използва се когато зависимостта е необходима при компилиране и изпълнение на тестовете.
- **system-** Подобно на „provided“ но се посочва наличен JAR наличен във файловата система.
- **systemPath** – използва се само заедно със „system“. Описва пътя до конкретния JAR.
- **Import** – Използва се вътре в „dependency management tag“. Добавя конфигурация със зависимости от друг проект.

ЗАВИСИМОСТ - ИЗКЛЮЧЕНИЯ

- Има ситуации в които не желаем да включваме всички транзитивни зависимости. Тогава добавяме конфигурация за изключение

```
<exclusion>  
  <groupId>*</groupId>  
  <artifactId>*</artifactId>  
</exclusion>
```

Изключва всички транзитивни зависимости

```
<dependencies>  
  <dependency>  
    <groupId>org.apache.maven</groupId>  
    <artifactId>maven-embedder</artifactId>  
    <version>2.0</version>  
    <exclusions>  
      <exclusion>  
        <groupId>org.apache.maven</groupId>  
        <artifactId>maven-core</artifactId>  
      </exclusion>  
    </exclusions>  
  </dependency>  
  ...  
</dependencies>
```

НАСЛЕДЯВАНЕ

- Едно от най – силните възможности на Maven е концепцията за наследяване на проекти. Типа на пакетирание трябва задължително да е „pom“.
- В резултат „децата“ наследяват ->

```
6.
7.  <groupId>org.codehaus.mojo</groupId>
8.  <artifactId>my-parent</artifactId>
9.  <version>2.0</version>
10. <packaging>pom</packaging>
```

- | | |
|-------------------|---------------------------------------|
| ▪ groupId | ▪ ciManagement |
| ▪ version | ▪ properties |
| ▪ description | ▪ dependencyManagement |
| ▪ url | ▪ dependencies |
| ▪ inceptionYear | ▪ repositories |
| ▪ organization | ▪ pluginRepositories |
| ▪ licenses | ▪ build |
| ▪ developers | ▪ plugin executions with matching ids |
| ▪ contributors | ▪ plugin configuration |
| ▪ mailingLists | ▪ etc. |
| ▪ scm | ▪ reporting |
| ▪ issueManagement | ▪ profiles |

НАСЛЕДЯВАНЕ - AGGREGATION (OR MULTI-MODULE)

- родител

```
7.  <groupId>org.codehaus.mojo</groupId>
8.  <artifactId>my-parent</artifactId>
9.  <version>2.0</version>
10. <packaging>pom</packaging>
11.
12. <modules>
13.   <module>my-project</module>
14.   <module>another-project</module>
15.   <module>third-project/pom-example.xml</module>
16. </modules>
17. </project>
```

- дете

```
6.
7.  <parent>
8.    <groupId>org.codehaus.mojo</groupId>
9.    <artifactId>my-parent</artifactId>
10.   <version>2.0</version>
11.   <relativePath>../my-parent</relativePath>
12. </parent>
13.
14.  <artifactId>my-project</artifactId>
15. </project>
```

MAVEN BUILD LIFECYCLE

- В Maven “действията” са цели, пакетирани в Maven плъгини, които са обвързани с фази в жизнения цикъл на компилацията.
- Жизненият цикъл на изграждането е организирана последователност от фази, които съществуват, за да се даде ред на набор от цели. Тези цели се избират и обвързват с типа пакетирание на проекта, по който се действа. В Maven има три стандартни жизнения цикъл: **clean**, **default** и **site**

DEFAULT LIFECYCLE

Lifecycle Phase	Описание
validate	Проверете дали проектът е правилен и има цялата необходима информация за завършване на компилация
generate-sources	Генерирайте всеки изходен код за включване в компилацията
process-sources	Обработва изходния код, например, за да се филтрират определени стойности
generate-resources	Генерира ресурси за включване в пакета.
process-resources	Копира и поставя ресурсите в директорията от която ще се пакетират
compile	Компилира кода на приложението
process-classes	Последваща обработка на генерираните файлове от компилацията(например да се подобри байт кода в Java класовете)
generate-test-sources	Генерира тестовия код, за добавяне в компилацията
process-test-sources	Обработва изходния тестов код, например, за да се филтрират определени стойности

DEFAULT LIFECYCLE

Lifecycle Phase	Описание
generate-test-resources	Създава ресурсите за тестването
process-test-resources	Копира и поставя тестовите ресурсите в целевата директория
test-compile	Компилирайте изходния тестов код в целевата тестова директория
test	Стартира тестовете
prepare-package	Подготвя пакетирането.
package	Взема компилирания код и го пакетира в определения формат–JAR,WAR...
pre-integration-test	Извършва необходимите операции за изпълнение на интеграционните тестове – примерно конфигуриране на средата за изпълнение
integration-test	Доставя необходимите пакети в средата, където интеграционните тестове могат да се изпълнят
post-integration-test	Изпълнява операцтите необходими след завършването на интеграционните тестове – почистване на средата

DEFAULT LIFECYCLE

Lifecycle Phase	Описание
verify	Старира и проверява валидността на пакетирания ресурс и дали отговарят на критериите за качество.
install	Инсталира пакетирания ресурс в локалното хранилище за използване от други приложения
deploy	Копира крайния пакетирания ресурс в отдалеченото хранилище за споделян с други програмисти и проекти(обикновено при релийз)

DEFAULT GOALS FOR JAR PACKAGING

Lifecycle Phase	Goal
process-resources	resources:resources
compile	compiler:compile
process-test-resources	resources:testResources
test-compile	compiler:testCompile
test	surefire:test
package	jar:jar
install	install:install
deploy	deploy:deploy

DEFAULT GOALS FOR POM PACKAGING

Lifecycle Phase	Goal
package	site:attach-descriptor
install	install:install
deploy	deploy:deploy

DEFAULT GOALS FOR WAR PACKAGING

Lifecycle Phase	Goal
process-resources	resources:resources
compile	compiler:compile
process-test-resources	resources:testResources
test-compile	compiler:testCompile
test	surefire:test
package	war:war
install	install:install
deploy	deploy:deploy

ПЛЪГИНИ

- Maven е инструмент базиран на плъгини и всичко се върши от тях. Всяка цел с която се стартира даден процес в Maven всъщност се стартира даден плъгин. Плъгините се разделят на Такива за изграждане на проекта и докладчици.

Plugin	Type*	Version	Release Date	Description
Core plugins				Plugins corresponding to default core phases (ie. clean, compile). They may have multiple goals as well.
clean	B	3.1.0	2018-04-14	Clean up after the build.
compiler	B	3.7.0	2017-09-04	Compiles Java sources.
deploy	B	2.8.2	2014-08-27	Deploy the built artifact to the remote repository.
failsafe	B	2.21.0	2018-03-06	Run the JUnit integration tests in an isolated classloader.
install	B	2.5.2	2014-08-27	Install the built artifact into the local repository.
resources	B	3.0.2	2016-12-10	Copy the resources to the output directory for including in the JAR.
site	B	3.7	2017-12-31	Generate a site for the current project.
surefire	B	2.21.0	2018-03-06	Run the JUnit unit tests in an isolated classloader.
verifier	B	1.1	2015-04-14	Useful for integration tests - verifies the existence of certain conditions.

ПЛЪГИНИ

Reporting plugins				Plugins which generate reports, are configured as reports in the POM and run under the site generation lifecycle.
<code>changelog</code>	R	2.3	2014-06-24	Generate a list of recent changes from your SCM.
<code>changes</code>	B+R	2.12.1	2016-11-01	Generate a report from an issue tracker or a change document.
<code>checkstyle</code>	B+R	3.0.0	2018-01-07	Generate a Checkstyle report.
<code>doap</code>	B	1.2	2015-03-17	Generate a Description of a Project (DOAP) file from a POM.
<code>docck</code>	B	1.1	2015-04-03	Documentation checker plugin.
<code>javadoc</code>	B+R	3.0.0	2017-12-06	Generate Javadoc for the project.
<code>jdeps</code>	B	3.1.1	2018-02-28	Run JDK's JDepends tool on the project.
<code>jxr</code>	R	2.5	2014-11-02	Generate a source cross reference.
<code>linkcheck</code>	R	1.2	2014-10-08	Generate a Linkcheck report of your project's documentation.
<code>pmd</code>	B+R	3.9.0	2018-01-21	Generate a PMD report.
<code>project-info-reports</code>	R	2.9	2016-03-01	Generate standard project reports.
<code>surefire-report</code>	B	2.21.0	2018-03-06	Generate a report based on the results of unit tests.

ЦЕЛ (GOAL) - ПРИМЕРИ

- `mvn install`
 - Изпълнява `generate*` и `compile`, `test`, `package`, `integration-test`, `install`
- `mvn clean`
 - Изпълнява `clean`
- `mvn clean compile`
 - Изчиства старите изграждания и изпълнява `generate*`, `compile`
- `mvn compile install`
 - Изпълнява `generate*`, `compile`, `test`, `integration-test`, `package`, `install`
- `mvn test clean`
 - Изпълнява `generate*`, `compile`, `test` след това изчиства

ЛИТЕРАТУРНИ ИЗТОЧНИЦИ

- <https://maven.apache.org>